

Compile-Time Java Stream Fusion via mapMulti

Yegor Bugayenko

yegor256@huawei.com

Huawei

Shenzhen, China

Innopolis University

Kazan, Russia

Maxim Trunnikov

trunnikov.maxim@huawei.com

Huawei

Shenzhen, China

Vladimir Zakharov

vladimir.zakharov@huawei.com

Huawei

Shenzhen, China

Abstract

The Java Stream API, introduced in Java 8, makes data processing more expressive and concise compared to imperative loops. However, this abstraction can come with significant performance overhead, often due to the creation of multiple intermediate objects during pipeline execution. In functional languages such as Haskell, this problem is addressed through stream fusion, a compile-time optimization that eliminates unnecessary intermediate structures. Inspired by this idea, STREAMLINER was the first tool to perform ahead-of-time, bytecode-to-bytecode stream optimization for Java by unrolling stream pipelines into imperative loops. In this paper, we introduce an open-source optimizer that takes a different approach. Instead of unrolling streams into loops, it merges consecutive `map()` and `filter()` operations into a single `mapMulti()` call, available since Java 16. Our method avoids several limitations of STREAMLINER, including its sensitivity to escaping objects in lambda expressions and its restrictions on assigning or passing streams as variables. We evaluated our optimizer on nine benchmarks and observed superior performance in two cases and comparable results in most others. We also applied it to the bytecode of Apache Kafka, successfully executing all 31,799 unit tests without failures.

CCS Concepts: • Software and its engineering → Preprocessors; • Computing methodologies → Algebraic algorithms; Symbolic calculus algorithms.

Keywords: Java, Stream API, Stream Fusion, Program Optimization, Static Analysis

ACM Reference Format:

Yegor Bugayenko, Maxim Trunnikov, and Vladimir Zakharov. 2026. Compile-Time Java Stream Fusion via `mapMulti`. In *Proceedings of the 15th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (SOAP '26)*, June 15–19, 2026, Boulder, CO, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3814987.3814988>



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOAP '26, Boulder, CO, USA

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2709-2/2026/06

<https://doi.org/10.1145/3814987.3814988>

'26), June 15–19, 2026, Boulder, CO, USA. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3814987.3814988>

1 Introduction

The Stream API¹ introduced in Java 8 makes stream processing more expressive [17], but its heavy reliance on virtual calls to temporary objects may make streams slower than imperative loops [16]. Stream fusion may help by reducing stream operations and virtual calls [13]. Consider the following Java stream pipeline:

```
1 var NN = new int[] {1, 1, 2, 3, 5, 8, 13};
2 var s = IntStream.of(NN)
3   .map(x -> x * x)
4   .map(x -> x + 1)
5   .sum();
```

It can be optimized by joining two `map` operations into one:

```
6 var s = IntStream.of(NN).map(
7   x -> x * x + 1).sum();
```

It can be further unrolled into an imperative loop:

```
8 var s = 0;
9 for (var i = 0; i < NN.length; i++)
10   s += NN[i] * NN[i] + 1;
```

Møller and Veileborg [21] introduced STREAMLINER, the first ahead-of-time (AOT) bytecode-to-bytecode optimizer. It *unrolls* some stream pipelines into loops, eliminating virtual calls. This approach has limitations, though: objects may not escape from lambda expressions, and streams may not be assigned to variables or passed as method arguments. Moreover, STREAMLINER increased performance in older JVMs, but we couldn't reproduce similar results in JVMs newer than 17; our experiments showed it may even *reduce* performance of short pipelines in recent JVM versions.

Java 16 extended the Stream API with the `mapMulti` method, which works similarly to `flatMap` but is an order of magnitude faster. We implemented stream *fusion* by combining pairs of `map` and `filter` operations into a single `mapMulti` operation, merging multiple stream operations into one rather than replacing streams with

¹<https://openjdk.org/projects/jdk8/>

loops. After our optimization, the snippet at lines 2–5 would look like this:

```
11 | var s = IntStream.of(NN).mapMulti(
12 |   (x, c) -> { c.accept(x * x + 1); }
13 | ).sum();
```

Instead of direct bytecode manipulation, our optimization pipeline is implemented as a rewriting system. We disassemble bytecode to φ -calculus expressions [4], rewrite them, and assemble into bytecode. The usage of φ -calculus enabled bytecode manipulation through declarative pattern matching and rewriting.

We released HONE to Maven Central, an open source Maven plugin that implements our method². Any Java project may optimize its bytecode after compilation by adding our plugin to their Maven or Gradle build configuration. The plugin requires no modifications to source code.

We asked the following research questions: **RQ1**: Can replacing sequences of `map` and `filter` operations with a single `mapMulti` operation provide performance gains similar to stream unrolling? **RQ2**: Can stream fusion via `mapMulti` be applied safely, not disrupting the functionality of existing stream pipelines?

Our results were as follows: On two of nine benchmarks used by Møller and Veileborg [21], our optimization outperformed STREAMLINER. On five benchmarks, our optimization matched STREAMLINER. Our method didn't optimize two benchmarks because they use terminal stream operations that our method is not intended to optimize. Optimization of Apache Kafka bytecode didn't cause any of its 31,799 unit tests in 1,511 test suites to fail.

Based on the results, we can provide the following answers to our RQs: First, stream fusion via `mapMulti` can provide performance gains comparable to or better than stream unrolling. Second, our optimization has fewer applicability limitations in production Java systems compared with STREAMLINER.

In summary, the contributions of this paper are: 1) We propose a formal intermediate representation for Java bytecode based on φ -calculus, enabling systematic stream fusion via rewrite rules and bridging low-level bytecode with high-level algebraic optimization; 2) We present JEO v0.15.1, a Maven plugin for bidirectional mapping between Java bytecode and φ -calculus expressions³; a rewriting system with 44 rules that replaces `map` and `filter` operations with `mapMulti` operations; PHINO v0.0.0.67, a tool that applies rewriting rules to φ -calculus expressions⁴; and HONE v0.24.0, a Maven plugin that automatically optimizes bytecode `.class` files

²<https://github.com/objectinary/hone-maven-plugin>

³<https://github.com/objectinary/jeo-maven-plugin>

⁴<https://github.com/objectinary/phino>

Table 1. The “Ratio” column shows stream time divided by loop time. It compares Fig. 1a with Fig. 1b for each JVM version. The numbers demonstrate that the performance of streams was improved since earlier JVM versions.

Vendor	JVM	Ratio	RSD	JVM	Ratio	RSD
Corretto	8.0.442	1.32	2.4%	24.0.2	1.01	4.9%
Zulu	8.0.462	1.32	2.0%	24.0.2	0.96	1.5%

after compilation; 3) We report an experimental evaluation showing that our method of stream fusion may be more effective on some benchmarks than STREAMLINER, especially on modern JVMs. 4) We demonstrate by experiment that our method doesn't disrupt the functionality of 31,799 unit tests in Apache Kafka.

All our digital artifacts, including the results of experiments and all used software, are published in Zenodo [?].

2 Background

Java 8 introduced lambda expressions and streams in 2014. Streams make code more expressive compared to traditional imperative loops [16]. Although they resemble functions from functional programming, they are objects, not functions. The Stream API consists of regular Java classes, while the lambda expressions passed to stream operations are compiled to `invokedynamic` instructions and corresponding static methods. At runtime, when the JVM encounters an `invokedynamic` instruction, it creates an instance of a class that implements the target functional interface. All calls to the lambda expression then become virtual calls to that created instance.

Streams in Java can be slow due to their abstraction overhead [16, 17]. Each `map` and `filter` operation creates a new stream with its own infrastructure. For a simple sum of five squares, the stream pipeline invokes the lambda function five times, plus additional overhead from stream machinery:

```
14 | var NN = {1, 2, 3, 4, 5};
15 | var s = IntStream.of(NN).map(x -> x *
    |   x).sum();
```

This algorithm can be rewritten as an imperative loop that doesn't use virtual calls:

```
16 | var s = 0;
17 | for (var i = 0; i < NN.length; i++)
18 |   s += NN[i] * NN[i];
```

Performance advantages of the imperative solution were obvious in earlier JVMs. However, modern JVMs are different.

```

19 public long loop() {
20     long sum = 0L;
21     for (int i=0; i<NN.length; ++i)
22         if (NN[i] % 2 == 0)
23             sum += NN[i] * NN[i];
24     return sum; }

```

(a) Imperative style.

```

25 public long map() {
26     return IntStream.of(NN)
27         .filter(x -> x % 2 == 0)
28         .map(x -> x * x)
29         .sum(); }

```

(b) Functional style, using a stream pipeline.

Figure 1. Two variants of computing sums of even squares suggested by Møller and Veileborg [21] to demonstrate that the usage of the Stream API degrades performance.

Modern JVMs handle streams better than their earlier versions. We reproduced the experiment of Møller and Veileborg [21]. Figure 1 shows two code snippets they tested with Java 13 using the OpenJDK Server VM (build 13+33). They observed a “60% performance degradation” for the stream approach (Fig. 1b) compared to the loop (Fig. 1a). Table 1 shows the data we collected: the degradation is present only in versions older than Java 11. Moreover, some of the latest JVMs calculate the sum of even numbers faster when using a stream of integers.

In this and all other experiments, we used JMH⁵ with two warming-up and five measuring iterations. We used a bare metal Ubuntu 24.04 at GHz x86_64 machine with 4 cores and 15 GiB of memory. The maximum measurement error observed in all experiments was 0.022%. The size of the NN array was ten million. The relative standard deviation (RSD), here and in all other tables in the paper, is a measure of relative variability. We performed all experiments two times and calculated RSD from the standard deviation and mean. In all our measurement experiments we used JVMs installable via SdkMan⁶.

Under the hood, modern HotSpot JVMs have dramatically improved their JIT compilers and runtime machinery compared to the Java 8 era.

First, their inlining is more aggressive. In Java 8, the C2 compiler’s default MaxInlineLevel was 9, limiting nested call inlining. By JDK 11, the inlining budget and heuristics were refined. They better handle functional patterns. Small lambdas and short stream pipeline operations are inlined and optimized into efficient loops.

Second, they improved escape analysis and scalar replacement. Intermediate objects created by the Stream API can now be optimized away. If the JIT proves that they never escape the method boundary, it eliminates the allocation entirely through scalar replacement. This transforms what used to be dozens of fleeting allocations and GC events into pure register operations.

By contrast, a manually written loop often presents less uniform control flow. It hides opportunities for scalar replacement and may not be profiled as comprehensively by the compiler. Short pipelines—once vilified for their “object churn”—now run as fast as or faster than equivalent for-loops.

3 Optimization Pipeline

To reason about and transform bytecode structurally, we encode it as φ -expressions, a minimal, uniform representation that exposes both instructions and metadata in a single composable form. The full calculus was introduced by Bugayenko and Trunnikov [4]; we use a subset sufficient to represent Java bytecode.

Figure 2 shows a stream pipeline compiled to bytecode and then disassembled into a φ -expression. Because φ -calculus uses composable symbolic structures, its expressions are amenable to rule-based rewriting. For example, the following rule transforms IMUL to IADD, where B_1 and B_2 match any sequence of key–value pairs:

$$\frac{\text{pattern}}{[B_1, \varphi \mapsto \Phi.\text{imul}, B_2]} \longrightarrow \frac{\text{replacement}}{[B_1, \varphi \mapsto \Phi.\text{iadd}, B_2]}.$$

Rules may use functions from PHINO, our rewriting engine. The following rule renames classes matching a pattern:

$$\frac{[B_1, \text{name} \mapsto e_1, B_2]}{\text{when } \text{matches}(\text{"^Nums\$"}, e_1) \text{ where } e_2 \models \text{concat}(\text{"Op"}, \text{name}).}$$

Rules are applied repeatedly until none match; order is irrelevant.

We scan bytecode files using the ASM library⁷ and disassemble them into φ -expressions (one .class file produces one .phi file).

Each instruction becomes a formation qualified by a dispatch (e.g., $\Phi.\text{class}$ for classes, $\Phi.\text{method}$ for methods); parameterized instructions like ILOAD carry their parameters as attributes. Java names are prefixed with $j\$$ to distinguish them from meta-attributes.

⁵<https://github.com/openjdk/jmh>

⁶<https://sdkman.io/>

⁷<https://asm.ow2.io/>

<pre> 30 import 31 java.util.stream.*; 32 class Numbers { 33 int total(int[] a) { 34 return IntStream 35 .of(a) 36 .map(x -> x * x) 37 .sum(); 38 } 39 } </pre>	<pre> 40 public Numbers(): 41 aload_0 42 invokespecial 43 return 44 public int 45 total(int[]): 46 aload_1 47 invokestatic 48 invokedynamic 49 invokeinterface 50 invokeinterface 51 ireturn 52 private static int 53 total\$0(int): 54 iload_0 55 iload_0 56 imul 57 ireturn </pre>	<pre> [[j\$Numbers ↦ [φ ↦ Φ.class, version ↦ 61, access ↦ 33, supertype ↦ "java/lang/Object", interfaces ↦ [], j\$object@init@ ↦ [...], j\$total ↦ [...], j\$total\$0 ↦ [φ ↦ Φ.method, access ↦ 4106, descriptor ↦ "(I)I", body ↦ [φ ↦ Φ.seq.of6, i2 ↦ [φ ↦ Φ.iload, v ↦ 0], i3 ↦ [φ ↦ Φ.iload, v ↦ 0], i4 ↦ [φ ↦ Φ.imul], i5 ↦ [φ ↦ Φ.ireturn]]]]]] </pre>
(a) Java class.	(b) Bytecode, from javap.	(c) Simplified φ -expression.

Figure 2. Java class after compilation and disassembly to φ -expression.

The Java compiler translates lambda expressions into `invokedynamic` and `invokeinterface` instruction pairs [26]. We rewrite these pairs into $\Phi.\text{filter}$ or $\Phi.\text{map}$ pragmas— φ -calculus formations that temporarily carry rewriting data but don’t exist in the final bytecode.

Both $\Phi.\text{filter}$ and $\Phi.\text{map}$ are then unified into $\Phi.\text{distill}$, an abstraction accepting an item and returning `void`. Consecutive $\Phi.\text{distill}$ pragmas are merged by concatenating their lambda bodies. Finally, $\Phi.\text{distill}$ pragmas become $\Phi.\text{mapMulti}$ pragmas, which generate new private static methods containing the merged logic plus a call to the `Consumer`’s `accept` method. The pragmas are then transformed back into instructions and reassembled into bytecode via ASM.

4 Results

RQ1: `mapMulti()` worked faster than `map()` + `filter()`. We used the same benchmark as Møller and Veileborg [21], since other Java benchmarks (SPECjbb 2015, DaCapo, Renaissance) either don’t use the Stream API or contain only trivial pipelines. Table 2 shows JMH measurements on two Zulu JVM versions; ratios below 1.0 indicate speedup.

On the latest JVM, three benchmarks (`sum`, `sumOfSquares`, `sumOfSquaresEven`) showed no improvement from either method. Two benchmarks (`megamorphic*`) ran faster with both. Two benchmarks (`allMatch`, `count`) improved only with STREAMLINER, as our method doesn’t modify these terminal operations. Notably, two benchmarks (`filterCount`,

`filterMapCount`) ran slower after STREAMLINER but showed no degradation with our method. Overall, our optimization either preserved speed or produced up to 5x improvements, with no performance degradation.

RQ2: The optimization didn’t disrupt existing functionality. We applied HONE to Apache Kafka (tag 4.0.0): 992k lines of Java, 142k lines of Scala, 31,799 unit tests. After disabling 13 flaky tests, the original build took 120 minutes; with HONE injected into `build.gradle`, it took 165 minutes and affected 15k `.class` files. All tests passed without additional modifications. We also integrated HONE into its own Maven release pipeline; after 50+ releases, no functionality was disrupted. The modified Kafka is available at: <https://github.com/objectnary/kafka>.

5 Related Work

Eliminating intermediate structures in functional programs, known as “deforestation” or fusion [13], has been studied since the late 1980s [31] and implemented in compilers such as GHC [10]. A common approach is deductive fusion [3, 5, 6, 11, 12, 20, 23, 28], which applies rewrite rules to produce optimized programs. Similar techniques have been applied to OCaml and Scala through staging [17], and recent work on SUPERFUSION [14] relies on supercompilation.

In object-oriented languages, Kiselyov et al. [17] proposed stream fusion via *strymonas* for Scala and OCaml, though it cannot be adapted to Java due to missing

Table 2. Optimization effect of STREAMLINER (“STRM” column) compared with our method (“HONE” column) on two Zulu JVM versions. Each ratio divides optimized time by original time.

Benchmark	Zulu 17.0.16				Zulu 23.0.2			
	STRM	±RSD	HONE	±RSD	STRM	±RSD	HONE	±RSD
<code>allMatch</code>	0.16	1.5%	1.00	1.5%	0.18	2.2%	1.00	4.4%
<code>count</code>	0.06	0.0%	1.00	0.0%	0.11	0.0%	1.00	0.0%
<code>filterCount</code>	1.07	1.2%	0.92	1.1%	1.06	0.1%	0.92	1.7%
<code>filterMapCount</code>	1.05	0.4%	0.93	1.9%	1.07	1.0%	0.91	0.2%
<code>megamorphicFilters</code>	0.04	3.1%	0.05	3.2%	0.04	1.0%	0.05	1.4%
<code>megamorphicMaps</code>	0.01	0.6%	0.04	0.4%	0.01	2.3%	0.01	1.1%
<code>sum</code>	0.98	1.7%	1.00	3.2%	0.99	0.3%	1.00	0.5%
<code>sumOfSquares</code>	0.66	0.3%	1.00	0.2%	1.05	2.4%	1.03	5.7%
<code>sumOfSquaresEven</code>	0.91	1.0%	0.97	1.0%	0.91	1.9%	0.97	0.8%

metaprogramming capabilities [18]. Ribeiro [25] introduced a Java library transforming streams into loops, but both approaches require switching from the standard Stream API. Khatchadourian et al. [15] developed an Eclipse plug-in for detecting parallelizable streams, observing 3.5x speedups, while Basso et al. [1] proposed stream parallelization for optimization. Møller and Veileborg [21] introduced compile-time stream unrolling via STREAMLINER, demonstrating improvements on benchmarks by Biboudis et al. [2]. Similar solutions exist for C# (Steno [22]) and F# (LinqOptimizer).

For bytecode manipulation, frameworks such as ASM, Byte Buddy, Javassist, and Apache Commons BCEL provide imperative APIs requiring manual traversal. Analysis-oriented tools translate bytecode into intermediate representations: Doop uses Datalog facts, WALA IR [27] provides SSA-style analysis, OPAL [8] and TACAI [24] offer three-address code, and Tai-e [29] focuses on static analysis. Soot [30] provides multiple IRs (Baf, Jimple, Shimple, Grimp) for transformation, while GraalVM [7] uses internal graph-based IRs for compilation. Krakatau offers textual bytecode encoding without formal semantics. All these tools rely on imperative transformation or analysis-oriented IRs without declarative rewriting support. In contrast, PHINO enables developers to specify rewriting rules that the system applies automatically, shifting effort from imperative traversal to concise, formally specified rewrites.

6 Limitations

Our method requires JDK 16 or newer due to `mapMulti`, is hardwired to the Stream API tested on JDK 24, and currently transforms only `map` and `filter` among the eleven intermediate operations (`dropWhile`, `flatMap`, `gather`, `limit`, `peek`, `skip`, `sorted`, `takeWhile`, etc.). We cannot fuse closures — lambdas capturing outer

variables — because bytecode places `ILOAD` and `ALOAD` instructions between stream operations, breaking the consecutive pattern our rewriting rules expect. Similarly, we cannot fuse functional objects (e.g., `new Square()` instead of lambdas) as they compile to `new/invokespecial` rather than `invokedynamic`. Code relying on Reflection to inspect bytecode structure may break, though such practice is uncommon. Line metadata may be damaged, affecting debugging.

Build time increased by 45 minutes (37%) for Apache Kafka; merging the current two-component architecture (Java+XSLT and Haskell) into a unified stack may improve this. We lack micro-level profiling (allocation counts, JIT inlining traces) to clarify the source of performance gains, and we did not compare with JIT-level optimizations such as Graal or HotSpot intrinsics.

Regarding validity: we tested with arrays of ten million `int` elements—results may differ with other sizes, types, or objects. The benchmark by Møller and Veileborg [21] has low complexity compared to real-world pipelines, lacking method references and autoboxing. We used a single JVM vendor and hardware configuration with JMH measurements, where JIT behavior variations could affect outcomes. While Apache Kafka covers nearly one million lines of code, it may not represent all Stream API usage patterns, limiting generalizability to other codebases.

7 Discussion

We found no conclusive evidence of performance gain from optimizing Apache Kafka: total test duration before and after optimization differed by less than 1% across ten runs lasting over two hours each, with individual test times fluctuating up to five times—consistent with Georges et al. [9] on measurement noise in benchmarking. In such a mature project (900,000+ lines, 20,000+ pull requests, 1,000+ contributors), developers likely avoid

the Stream API in performance-critical paths, knowing that pipelines are generally slower than imperative loops.

Stream pipelines improve expressiveness and readability, which correlates with higher maintainability and fewer defects—yet these advantages are often sacrificed for performance. As Apache Cassandra developers noted: “we quickly realized that Streams and Lambdas were pretty bad from a performance point of view; due to this fact, we stopped using them in hot path” [19]. Compile-time optimizations like HONE or STREAMLINER may reconcile this trade-off, enabling programmers to adopt streams more widely while retaining acceptable performance.

While bytecode modifications could be done without an IR (e.g., STREAMLINER uses ASM directly), φ -calculus enabled us to express transformations declaratively through rewriting rules, eliminating control-flow boilerplate and reducing inconsistent edits. Its uniform, formally defined structure for all bytecode entities made the pipeline easier to reason about and extend—new optimizations require only new rewriting rules, written in the same surface syntax as the IR itself, without any modification to the rewriting engine or to JEO and PHINO. This shift from imperative engineering to a declarative, rule-based system simplified implementation and improved correctness reasoning. It is important to emphasize that φ -calculus [4] was not designed for this particular experiment: it is a generic formalism for object-oriented languages and is not tied to Java or to the JVM bytecode. Its expressions describe objects, attributes, and application in a language-agnostic way, and the mapping to Java bytecode performed by JEO is only one possible front-end; rewriting rules operate on φ -calculus expressions and are therefore reusable across any source language that can be translated into this IR.

Our focus on `map` and `filter` is intentional: they form a minimal yet practically relevant subset of the Stream API, and they are the two operations for which `mapMulti` provides a natural fusion target with favorable performance characteristics on modern JVMs. The goal of this study was not to cover the full surface of the Stream API, but to investigate whether fusion via `mapMulti` is a viable alternative to loop unrolling on recent JVMs, where AOT unrolling no longer yields consistent gains. The rewriting pipeline itself is not restricted to these two operations: because φ -calculus expressions uniformly represent all bytecode entities and transformations are expressed as rewriting rules, supporting additional intermediate operations—such as `reduce`, `flatMap`, `peek`, or `takeWhile`—amounts to adding new rules rather than changing the engine. We view the present work as a proof of concept for this broader program of φ -calculus-based stream optimization, and leave a systematic extension to the remaining intermediate operations for future work.

The absolute numbers—matching STREAMLINER on most micro-benchmarks, outperforming it on two, and yielding less than 1% on Apache Kafka—should be read as evidence that the method is competitive and safe, not as the main contribution. Our primary contribution is the method itself: a uniform, declarative φ -calculus-based IR in which transformations of object-oriented bytecode are expressed and composed as rewriting rules, instead of being hard-coded as ad-hoc pattern matches over raw instructions. The `map/filter-to-mapMulti` optimizer is one concrete instantiation, chosen because it is non-trivial to express with traditional bytecode-rewriting frameworks yet small enough to evaluate end-to-end; the same infrastructure can host more aggressive transformations with larger potential payoff.

8 Conclusion

Stream unrolling is a powerful but intrusive technique; in modern JVMs, stream fusion via `mapMulti` may provide comparable benefits with less intrusion. Our method differs from existing approaches by introducing a formal calculus for bytecode semantics: we map bytecode to well-defined structures, apply algebraic rewriting rules, and regenerate optimized bytecode. Compared to STREAMLINER [21] on Zulu JVM 23, two benchmarks took twice as long after their optimization while ours caused no degradation, two benchmarks gained equal performance from both methods, and two benchmarks using `count` and `allMatch` could only be optimized by STREAMLINER as these operations are outside our scope. We validated correctness on 31,799 Apache Kafka unit tests without disruption.

Acknowledgements

The work was sponsored by Huawei. We are grateful to Nikolai Kudasov for pointing our research into the direction of applying φ -calculus to solving the problem of stream fusion in Java.

No parts of the paper were written with the help of AI. Claude Code was used to fix grammar mistakes.

References

- [1] Matteo Basso, Filippo Schiavio, Andrea Rosà, and Walter Binder. 2022. Optimizing Parallel Java Streams. In *Proceedings of the 26th International Conference on Engineering of Complex Computer Systems*. 23–32. doi:10.1109/ICECCS5421.0.2022.00012
- [2] Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. 2014. Clash of the Lambdas. arXiv:1406.6631 [cs.PL]
- [3] R. S. Bird. 1989. Algebraic Identities for Program Calculation. *The Computer Journal* 32, 2 (1 1989), 122–126. doi:10.1093/comjnl/32.2.122
- [4] Yegor Bugayenko and Maxim Trunnikov. 2021. φ -Calculus: Object-Oriented Formalism. arXiv:2111.13384 [cs.PL]

- [5] Wei-Ngan Chin. 1992. Safe Fusion of Functional Expressions. In *Proceedings of the Conference on LISP and Functional Programming*. 11–20. doi:10.1145/141471.141494
- [6] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. 2007. Stream Fusion: From Lists to Streams to Nothing at All. *SIGPLAN Notices* 42, 9 (10 2007), 315–326. doi:10.1145/1291220.1291199
- [7] Gilles Duboscq, Lukas Stadler, Thomas Würthinger, Doug Simon, Christian Wimmer, and Hanspeter Mössenböck. 2013. Graal IR: An Extensible Declarative Intermediate Representation.
- [8] Michael Eichberg and Ben Hermann. 2014. A Software Product Line for Static Analyses: The OPAL Framework. In *Proceedings of the 3rd SIGPLAN International Workshop on the State of the Art in Java Program Analysis*. 1–6. doi:10.1145/2614628.2614630
- [9] Andy Georges, Dries Buytaert, and Lieven Eeckhout. 2007. Statistically Rigorous Java Performance Evaluation. *ACM SIGPLAN Notices* 42, 10 (2007), 57–76. doi:10.1145/1297105.1297033
- [10] Andrew Gill, John Launchbury, and Simon L. Peyton Jones. 1993. A Short Cut to Deforestation. In *Proceedings of the Conference on Functional Programming Languages and Computer Architecture*. 223–232. doi:10.1145/165180.165214
- [11] G. W. Hamilton. 2002. Extending Higher-Order Deforestation: Transforming Programs to Eliminate Even More Trees.
- [12] Ralf Hinze, Thomas Harper, and Daniel W. H. James. 2011. Theory and Practice of Fusion. In *Proceedings of the Implementation and Application of Functional Languages*. 19–37. doi:10.1007/978-3-642-24276-2_2
- [13] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. 2014. A Catalog of Stream Processing Optimizations. *ACM Computing Surveys* 46, 4 (2014), 1–34. doi:10.1145/2528412
- [14] Ruyi Ji, Yuwei Zhao, Nadia Polikarpova, Yingfei Xiong, and Zhenjiang Hu. 2024. Superfusion: Eliminating Intermediate Data Structures via Inductive Synthesis. *Proceedings of ACM Programming Languages* 8, 1 (6 2024). doi:10.1145/3656415
- [15] Raffi Khatchadourian, Yiming Tang, and Mehdi Bagherzadeh. 2020. Safe Automated Refactoring for Intelligent Parallelization of Java 8 Streams. *Science of Computer Programming* 195, 1 (2020). doi:10.1016/j.scico.2020.102476
- [16] Raffi Khatchadourian, Yiming Tang, Mehdi Bagherzadeh, and Baishakhi Ray. 2020. An Empirical Study on the Use and Misuse of Java 8 Streams. In *Proceedings of the International Conference on Fundamental Approaches to Software Engineering*. 97–118. doi:10.1007/978-3-030-45234-6_5
- [17] Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. 2017. Stream Fusion, to Completeness. *SIGPLAN Notices* 52, 1 (1 2017), 285–299. doi:10.1145/3093333.3009880
- [18] Oleg Kiselyov, Tomoaki Kobayashi, and Nick Palladinos. 2024. Complete Fusion for Stateful Streams: Equational Theory of Stateful Streams and Fusion as Normalization-by-Evaluation. arXiv:2412.15768 [cs.PL]
- [19] Davood Mazinanian, Ameya Ketkar, Nikolaos Tsantalis, and Danny Dig. 2017. Understanding the Use of Lambda Expressions in Java. *Proceedings of the ACM on Programming Languages* 1, 1 (2017), 1–31. doi:10.1145/3133909
- [20] Erik Meijer, Maarten Fokkinga, and Ross Paterson. 1991. Functional Programming With Bananas, Lenses, Envelopes and Barbed Wire. In *Proceedings of the Functional Programming Languages and Computer Architecture*. 124–144. doi:10.1007/3540543961_7
- [21] Anders Møller and Oskar Haarklou Veileborg. 2020. Eliminating Abstraction Overhead of Java Stream Pipelines Using Ahead-of-Time Program Optimization. In *Proceedings of the Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 1–29. doi:10.1145/3428236
- [22] Derek Gordon Murray, Michael Isard, and Yuan Yu. 2011. Steno: Automatic Optimization of Declarative Queries. In *Proceedings of the 32nd Conference on Programming Language Design and Implementation*. 121–131. doi:10.1145/1993498.1993513
- [23] Erik Poll and Simon Thompson. 1999. Algebra of Programming by Richard Bird and Oege De Moor. *Journal of Functional Programming* 9, 3 (1999), 347–354. doi:10.1017/S09567968992326X
- [24] Michael Reif, Florian Kübler, Dominik Helm, Ben Hermann, Michael Eichberg, and Mira Mezini. 2020. TACAI: An Intermediate Representation Based on Abstract Interpretation. In *Proceedings of the 9th SIGPLAN International Workshop on the State of the Art in Program Analysis*. 2–7. doi:10.1145/3394451.3397204
- [25] Francisco José Torres Ribeiro. 2018. Java Stream Optimization Through Program Fusion. <https://hdl.handle.net/1822/59688>. [Online; accessed 23-07-2025].
- [26] John R. Rose. 2009. Bytecodes Meet Combinators: Invoke-dynamic on the JVM. In *Proceedings of the Third Workshop on Virtual Machines and Intermediate Languages*. 1–11. doi:10.1145/1711506.1711508
- [27] Joanna C. S. Santos and Julian Dolby. 2022. Program Analysis Using WALA. In *Proceedings of the 30th Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1819. doi:10.1145/3540250.3569449
- [28] Akihiko Takano and Erik Meijer. 1995. Shortcut Deforestation in Calculational Form. In *Proceedings of the 7th International Conference on Functional Programming Languages and Computer Architecture*. 306–313. doi:10.1145/224164.224221
- [29] Tian Tan and Yue Li. 2023. Tai-e: A Developer-Friendly Static Analysis Framework for Java by Harnessing the Good Designs of Classics. In *Proceedings of the 32nd International Symposium on Software Testing and Analysis*. 1093–1105. doi:10.1145/3597926.3598120
- [30] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundareshan. 1999. Soot: A Java Bytecode Optimization Framework. In *Proceedings of the Conference of the Centre for Advanced Studies on Collaborative Research (CASCON'99)*. 214–224. doi:10.5555/781995.782008
- [31] Philip Wadler. 1990. Deforestation: Transforming Programs to Eliminate Trees. *Theoretical Computer Science* 73, 2 (1990), 231–248. doi:10.1016/0304-3975(90)90147-A

Received 2026-02-10; accepted 2026-04-15